

GIANI KARTAR SINGH MEMORIAL
GOVERNMENT COLLEGE, URMAR TANDA.

PROJECT REPORT

ON

COMPLAINT MANAGEMENT SYSTEM

Submitted in partial fulfilment of the academic requirements

for B.Sc. Non-Medical

SUBMITTED BY

Gurleen Saini

Roll Number: 24053394

PROJECT GUIDE

Miss Nishtha

PROJECT SUPERVISOR

Prof. Rajandeep Kaur

5th Semester | 3rd Year

Academic Session: 2026–2027

DECLARATION

I, Gurleen Saini, Roll Number 24053394, a student of B.Sc. Non-Medical, 5th Semester, 3rd Year, at Giani Kartar Singh Memorial Government College, Urmar Tanda, declare that the project report entitled “Complaint Management System” has been prepared by me as an individual academic project.

The report describes the design and development of a desktop application using Python, CustomTkinter and SQLite. Wherever implementation details or testing outcomes require confirmation against the final program, the report identifies them for verification. Any external material consulted for the project should be acknowledged in the final submitted version.

I understand that the report must represent my own project work and that I am responsible for checking the accuracy of the content, inserting genuine screenshots, and following the college’s submission guidelines.

Place: _____

Date: _____

Student Signature: _____

CERTIFICATE

This is to certify that the project report entitled “Complaint Management System” is submitted by Gurleen Saini (Roll Number 24053394), a student of B.Sc. Non-Medical, 5th Semester, 3rd Year, Giani Kartar Singh Memorial Government College, Urmar Tanda as an individual academic project.

The work has been prepared under the guidance of Miss Nishtha and the supervision of Prof. Rajandeep Kaur. The report is intended to document the project’s objectives, requirements, design, implementation approach and testing plan. The final version should be reviewed and approved by the designated college authorities before submission.

Project Guide: Miss Nishtha

Signature: _____

Project Supervisor: Prof. Rajandeep Kaur

Signature: _____

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Miss Nishtha, for her guidance, encouragement and support during the development of this Complaint Management System. Her suggestions helped me approach the project in a more organised way and improve my understanding of Python programming and graphical user-interface development.

I am also thankful to my project supervisor, Prof. Rajandeep Kaur, for supervising the work and providing academic direction. I extend my thanks to Giani Kartar Singh Memorial Government College for providing the academic environment in which I could undertake this project.

Finally, I would like to acknowledge the Python and db Browser (SQLite) documentation and learning resources that helped me understand the tools used in the application. This project has been a useful learning experience in programming, database handling, interface design and software testing.

PREFACE

The Complaint Management System is a desktop application designed to organise the recording and review of complaints in a structured way. In many small institutions, complaints may be communicated verbally, through paper forms, or through scattered messages. Such methods can make it difficult to locate earlier complaints, check their current status, and maintain consistent records.

This project applies basic software-development principles to create a graphical application in Python. CustomTkinter is used to build the desktop interface, while SQLite is used as a local database for storing complaint records. The application's intended workflow includes entering complaint information, viewing complaint records, searching and filtering the list, and reviewing management fields such as status, assigned officer and remarks.

The report follows the broad chapter structure of the supplied sample report: introduction, methodology, analysis, design, development, testing/results, implementation evaluation, conclusion, appendices and bibliography. Screenshots and test outcomes should be updated to match the final working version of the application.

TABLE OF CONTENTS

Section	Contents
Preliminary Pages	Declaration, Certificate, Acknowledgement, Preface
Chapter 1	Introduction
1.1	Problem Definition
1.2	Objectives and Scope
1.3	Minimum Requirements
1.4	Project Deliverables
1.5	Project Schedule
1.6	Relevance to the Degree Program
Chapter 2	Project Methodology
2.1	Development Methodologies
2.2	Selected Methodology
2.3	Development Phases
Chapter 3	Project Analysis
3.1	Intended Users and Existing Practices
3.2	Feasibility Study
3.3	Functional Requirements
3.4	Non-Functional Requirements
3.5	Hardware and Software Requirements
Chapter 4	Project Design
4.1	System Architecture
4.2	System Flowchart
4.3	Data Design
4.4	Interface Design
4.5	Decision Table
Chapter 5	Project Development
5.1	Tools and Environment
5.2	Implementation Overview
5.3	Database Operations
5.4	Interface Modules
5.5	Screenshots to Insert
Chapter 6	Project Testing and Results
6.1	Testing Approach
6.2	Test Cases
6.3	Verification and Validation
Chapter 7	Implementation Evaluation
7.1	Evaluation
7.2	Limitations
7.3	Future Scope
Chapter 8	Conclusion
Appendix A	User Guide
Appendix B	Sample Complaint Record
Bibliography	References and Documentation

CHAPTER 1: INTRODUCTION

1.1 Introduction

A complaint is a formal expression of a problem, difficulty or dissatisfaction that requires attention. In an educational institution, complaints may relate to academics, hostel facilities, fees, infrastructure, maintenance or other student services. A clear process for recording and reviewing complaints can help staff understand the issue, keep records organised and monitor progress.

The Complaint Management System developed for this project is a Python-based desktop application. Its interface is built with CustomTkinter, a library that provides themed widgets for Tkinter applications. SQLite is used as the local relational database. The system aims to provide a single place to record complaint details and review them through a graphical interface.

The current project is designed primarily as a local desktop application for demonstration and academic use. It is not presented as a live college-wide service, and it does not automatically contact departments or notify students unless such features are added separately.

1.2 Problem Definition

When complaints are recorded using paper notes, informal conversations or separate files, information can become scattered. Searching for a particular complaint may take time, and it may be difficult to identify whether an issue is pending, in progress or resolved. Records may also lack a consistent structure.

The problem addressed by this project is the need for a simple and organised way to enter complaint information, store it in a database, find existing records, and review their status and related administrative information through a desktop interface.

1.3 Objectives and Scope

The main objective is to develop a user-friendly desktop system that supports structured complaint record management.

- Provide a graphical interface for entering complaint details.
- Store complaint records in an SQLite database so they can be retrieved later.
- Display complaint records in a list and show the details of a selected complaint.
- Provide search and filtering controls for complaint ID, student name, subject, status and category, as implemented in the final build.
- Represent complaint status using values such as Pending, In Progress and Resolved.
- Provide interface fields for assigning an officer or department and entering administrative remarks, subject to final verification of save/update behaviour.
- Keep the interface readable through grouped panels, clear labels, colour-coded status indicators and scrollable sections.

Scope: The project focuses on the core complaint-record workflow on one computer. Multi-user access, web/mobile access, email/SMS alerts, online deployment, role-based permissions and automated analytics are outside the confirmed scope of the current version unless they are separately implemented.

1.4 Minimum Requirements

- A computer capable of running a supported Python installation.

- Python installed with CustomTkinter available.
- SQLite support, normally provided by Python's sqlite3 standard-library module.
- A desktop operating system with a graphical environment.
- Sufficient storage for the Python source files and local database.
- A user who can enter accurate complaint details and operate the interface.

1.5 Project Deliverables

Deliverable	Description
Python source code	Application windows and event-handling logic.
Database file	Local SQLite database containing complaint records and related fields used by the application.
Graphical interface	Login/sign-in and main application windows, Add Complaint window, and View Complaint window as implemented.
Project report	Documentation of objectives, methodology, analysis, design, development, testing and evaluation.
Screenshots and test evidence	Genuine screenshots and observed test outcomes to be added from the final application.

1.6 Project Schedule

Phase	Main Activities	Status / Evidence
1. Planning	Identify the complaint-management problem and decide the main modules.	Documented in this report.
2. Interface design	Create login/main screens and complaint windows.	Confirm final appearance with screenshots.
3. Database setup	Create SQLite connection and complaint table(s).	Verify schema and persistence in final code.
4. Core integration	Connect Add Complaint records to the View Complaint list.	User previously observed newly submitted records in the list; retest after final changes.
5. Search and review	Add search, filters and selected-complaint details.	Test each filter and selection action.
6. Administration fields	Status, assignment and remarks controls.	Confirm that values save to the database, not only the interface.
7. Testing and report	Run test cases, capture screenshots and prepare submission.	Complete before final submission.

1.7 Relevance to the Degree Programme

Although the project is an application-development task, it reinforces concepts useful in a science degree that includes computational or quantitative learning: logical reasoning, step-by-step problem solving, structured data, algorithmic thinking and verification of results. The project also provides practical experience in translating a real-world problem into requirements, a user interface, a database and testable operations.

CHAPTER 2: PROJECT METHODOLOGY

2.1 Introduction

A software-development methodology provides an organised way to plan, design, build, test and improve an application. For a small academic project, the methodology should be simple enough to follow while allowing changes as the interface and database are refined.

2.2 Common Development Methodologies

Methodology	Description	Suitability for this project
Waterfall	Work proceeds through defined stages, with each stage largely completed before the next.	Useful for a fixed-scope project, but less flexible when interface changes are frequent.
Prototyping	A preliminary version is created to collect feedback and refine requirements.	Useful for checking the appearance and flow of complaint windows early.
Iterative / Incremental	The system is developed in small parts, each adding or improving functionality.	Well suited to adding the main window, Add Complaint, View Complaint and database operations step by step.
RAD	Rapid Application Development emphasises quick prototyping and frequent feedback.	Some principles are useful for a small desktop application, though formal RAD practices may be unnecessary.

2.3 Selected Methodology

An iterative development approach is the most suitable description of the work completed for this project. The interface was developed in stages, and individual parts were refined after they were run and checked. For example, the main window and complaint windows were built separately; the View Complaint layout was then refined, and database integration was addressed after the basic visual structure was in place.

This description is based on the project workflow and should not be interpreted as a formal institutional SDLC certification. The approach is practical for a student project because it allows interface and functionality problems to be found early.

2.4 Development Phases

1. Requirement identification: list the complaint information and basic actions needed.
2. Interface planning: organise the main controls and decide how the user moves between windows.
3. Database planning: identify the fields required to store and retrieve complaint records.
4. Implementation: write Python functions and CustomTkinter widgets.
5. Integration: connect the Add Complaint workflow with the SQLite database and View Complaint window.
6. Testing: run normal, empty-input, search, filter, selection and persistence checks.
7. Evaluation: record limitations and identify realistic improvements.

2.5 Why the Approach Was Appropriate

The iterative approach made it possible to work on visual design before connecting every control to database operations. It also made debugging more manageable because layout, window behaviour, database access and event callbacks could be checked in smaller steps. A remaining requirement is to retest the complete application after all modules have been integrated.

3.1 Intended Users

The intended users are people who need to record or review complaints in a small organisational setting. Depending on the permissions and workflow established by the institution, these may include a person entering a complaint and an authorised staff member reviewing or updating it. The current project should be treated as a desktop prototype unless it has been formally deployed and secured for real institutional use.

3.2 Existing Practices and Need for the System

Possible existing practices include paper forms, notebooks, spreadsheets, email messages or verbal reporting. These approaches may work for a small number of cases, but records can be harder to search when information is stored in multiple places. A structured database and a common interface can reduce duplicate manual effort and make complaint details easier to retrieve.

The project does not assume that the college currently uses any particular complaint procedure. It demonstrates a possible digital workflow rather than making claims about the institution's existing systems.

3.3 Feasibility Study

Feasibility Type	Assessment
Technical	Python, CustomTkinter and SQLite are suitable for a local desktop prototype. Python provides the sqlite3 module for database access.
Economic	The principal tools used are available without a paid software licence. The main cost is development time and access to a computer.
Operational	A form-based interface can be used by a person familiar with basic desktop applications. Clear labels and validation are important.
Schedule	The project can be built incrementally. Final testing, screenshots and documentation still require time.
Maintenance	Source code and the local database can be maintained on the same computer. Backups are important because the database is stored locally.

3.4 Functional Requirements

ID	Requirement	Verification Note
FR-01	The application should provide a way to enter complaint information.	Check the Add Complaint form and required fields.
FR-02	Complaint records should be stored in a database.	Close and reopen the application; check whether the record remains.
FR-03	The View Complaint screen should display available complaint records.	Compare the displayed list with database records.
FR-04	A user should be able to select a complaint and view its details.	Select several records and compare the details.
FR-05	Search should help locate records by supported text fields.	Test ID, student name and subject separately.
FR-06	Status and category filters should narrow the displayed records.	Test each filter and the All option.
FR-07	The interface should offer status choices such as Pending, In Progress and Resolved.	Confirm whether selection is saved to SQLite.

FR-08	The interface should offer officer/department assignment and remarks fields.	Confirm persistence before describing them as fully operational.
FR-09	The application should provide clear feedback for invalid or incomplete input.	Check actual validation and error messages in the final build.

3.5 Non-Functional Requirements

- Usability: labels and controls should be understandable and grouped logically.
- Reliability: database operations should handle connection and query errors without silently losing records.
- Performance: the application should remain responsive for a modest number of local complaint records.
- Maintainability: functions and database operations should be separated clearly where practical.
- Data integrity: complaint identifiers and required fields should be handled consistently.
- Privacy: complaint data should be accessible only to authorised people; the current local prototype needs additional access controls before real deployment.
- Portability: the project should run on a computer with a compatible Python environment and required packages.

3.6 Hardware and Software Requirements

Type	Minimum / Expected Requirement
Hardware	Desktop or laptop computer; keyboard and mouse/touchpad; enough storage for Python, project files and the database.
Operating system	Windows or another desktop OS supported by the installed Python and CustomTkinter environment.
Programming language	Python 3.x; use the version verified on the development computer.
GUI library	CustomTkinter, with Tkinter support available.
Database	SQLite, accessed through Python's sqlite3 module.
Editor / IDE	Visual Studio Code was used during development, according to the project workflow.
Other	A compatible Python interpreter and any additional packages imported by the actual source code.

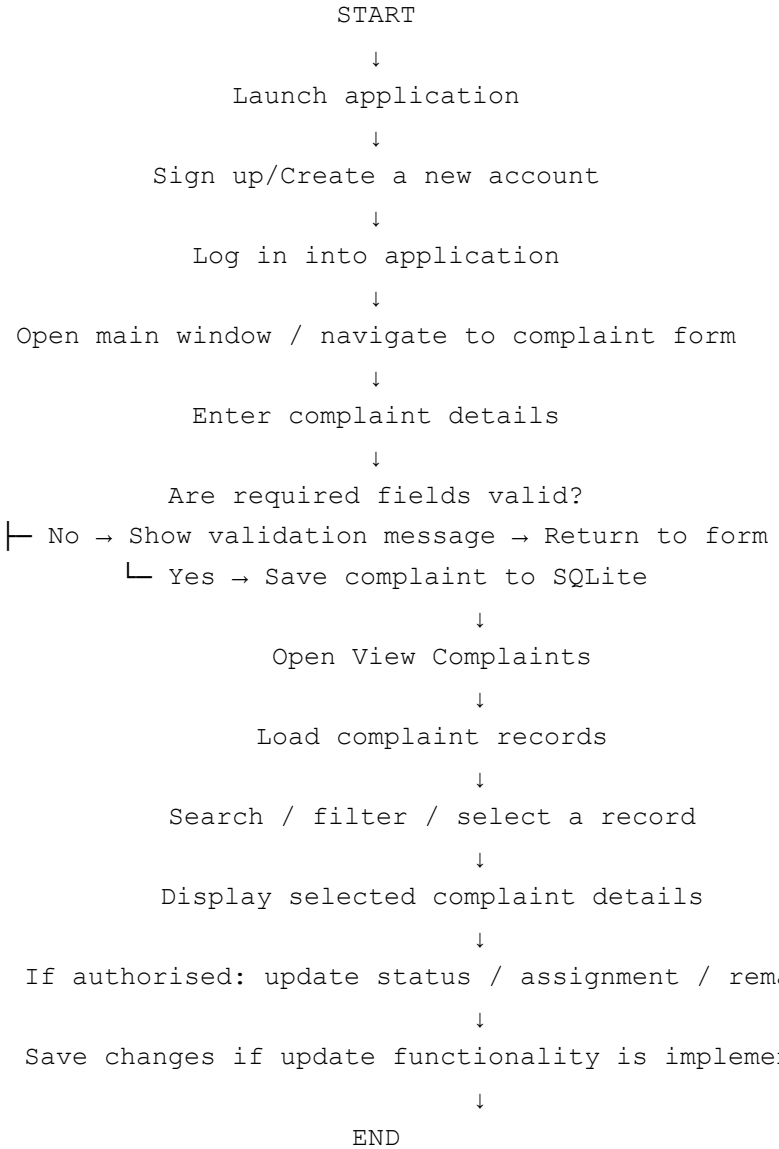
CHAPTER 4: PROJECT DESIGN

4.1 System Architecture

The application follows a simple local desktop architecture. The user interacts with CustomTkinter windows. Python event handlers respond to button clicks, selections and form submissions. Database helper functions open an SQLite connection, execute queries and return records to the interface.

Layer	Role
Presentation layer	CustomTkinter windows, labels, forms, buttons, scrollable frames, status badges and filters.
Application logic	Functions that validate input, handle events, select a complaint, filter results and coordinate window navigation.
Data layer	SQLite database file and SQL statements used to create, insert, select and update complaint records.

4.2 System Flowchart



Note: The flowchart represents the intended workflow. The final code must be checked to confirm validation, persistence of administrative changes and the exact navigation sequence.

4.3 Data Design

A complaint record should contain enough information to identify the issue, understand it and track its progress. The following table is a conceptual field list; the actual database column names and types must be matched to the project's SQLite schema.

Field (conceptual)	Purpose	Example / Notes
complaint_id	Unique identifier for a complaint.	Example display ID: CMS-1024.
submission_date	Date the complaint was submitted.	Store consistently, preferably in ISO format internally.
student_name	Name of the person submitting the complaint.	Use accurate spelling.
roll_number	Student or institutional identifier.	Treat as text if leading zeros or letters may occur.
subject	Short title of the issue.	Example: Hostel Room Maintenance.
category	Broad complaint type.	Hostel, Academics, Fee/Finance, Infrastructure.
description	Detailed explanation of the issue.	Text field.
status	Current workflow state.	Pending, In Progress, Resolved.
assigned_to	Officer or department responsible.	If supported by the final schema.
remarks	Administrative notes or feedback.	If supported by the final schema.
updated_at / updated_by	Optional audit information.	Only include if implemented in the database.

4.4 Interface Design

The interface uses a blue-and-white visual theme to distinguish the title area, content panels, filters and complaint details. In the View Complaint window, a header introduces the page; a filter area provides search and dropdown controls; the main area contains the complaint list on one side and selected complaint details on the other. Scrollable frames help keep the screen usable when complaint details or the list become longer.

- Main window: navigation to core complaint actions.
- Add Complaint window: form for entering a complaint record.
- View Complaint window: searchable list, status/category filters and detail panel.
- Complaint card: compact summary with complaint ID, date, category, status and student information.
- Details panel: full description and related information for the selected complaint.
- Administrative area: controls for status, assignment and remarks; verify that each control is connected to save/update logic.

4.5 Decision Table

Condition / Action	Required fields valid?	Record exists?	Expected behaviour
Submit complaint	Yes	Not applicable	Attempt to save a new complaint and provide success feedback if save succeeds.
Submit complaint	No	Not applicable	Prevent or reject incomplete input and explain what needs correction.
View complaint	Not applicable	Yes	Display the selected complaint's stored details.

View complaint	Not applicable	No	Show an empty state or message rather than crashing.
Apply search/filter	Not applicable	Matching records exist	Display only records matching the selected criteria.
Apply search/filter	Not applicable	No matching records	Show a clear no-results state.
Update administrative fields	Not applicable	Yes	Save changes to the database if the update feature is implemented; otherwise do not claim persistence.

CHAPTER 5: PROJECT DEVELOPMENT

5.1 Development Tools and Environment

The project is developed in Python and uses CustomTkinter for the desktop graphical user interface. The sqlite3 module provides access to the SQLite database. Visual Studio Code is the development editor used in the project workflow. Exact package and interpreter versions should be recorded from the development computer before final submission.

5.2 Python

Python is a high-level programming language with readable syntax and a large standard library. In this project, Python manages the window structure, event callbacks, validation, database access and display of complaint records. Its sqlite3 module allows SQL commands to be executed without installing a separate database server.

5.3 CustomTkinter

CustomTkinter extends Tkinter with themed widgets and appearance options. The project uses widgets such as CTk, CTkToplevel, CTkFrame, CTkLabel, CTkButton, CTkEntry, CTkOptionMenu, CTkTextbox and CTkScrollableFrame where appropriate. Frames organise related controls, buttons trigger actions, and scrollable frames allow content to extend beyond the visible area.

5.4 SQLite Database

SQLite stores data in a local database file. The application can use a helper function to open a connection, create tables if needed, insert a new complaint and retrieve records for display. Connections and cursors should be managed carefully, and changes should be committed after successful write operations. Queries should use parameter placeholders rather than joining user input directly into SQL strings.

A typical conceptual workflow is: open a connection, execute a parameterised query, commit if data changed, handle exceptions, and close the connection. This paragraph describes good practice; verify that the actual source code follows it.

5.5 Implementation Overview

Module / Component	Purpose	Current report treatment
Login / sign-in interface	Provides an entry point to the application.	Include only the behaviour that is present in the final code.
Main window	Provides navigation to complaint actions.	Buttons have been planned for Add, Update, Delete and View operations.
Add Complaint window	Collects the complaint’s details.	Verify required-field checks and save confirmation.
View Complaint window	Displays complaint records and selected details.	Includes search/filter layout and a complaint list/detail area.
Database helper	Creates connections and performs SQL operations.	Database file and table names should match the code.
Administrative controls	Status, officer/department assignment and remarks.	Verify each operation updates SQLite before describing it as complete.

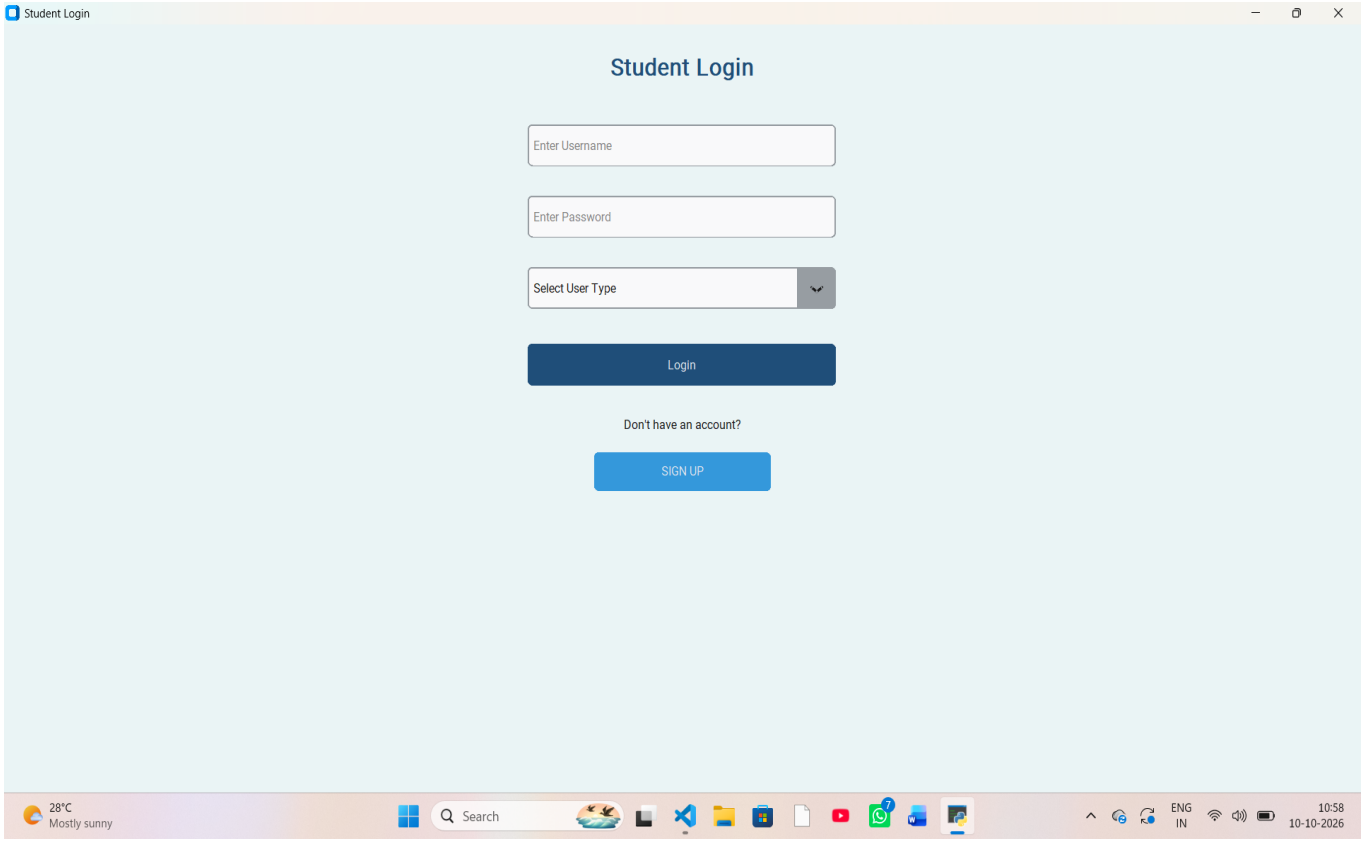
5.6 Database Operations

The database operations expected in a complaint-record application include the following:

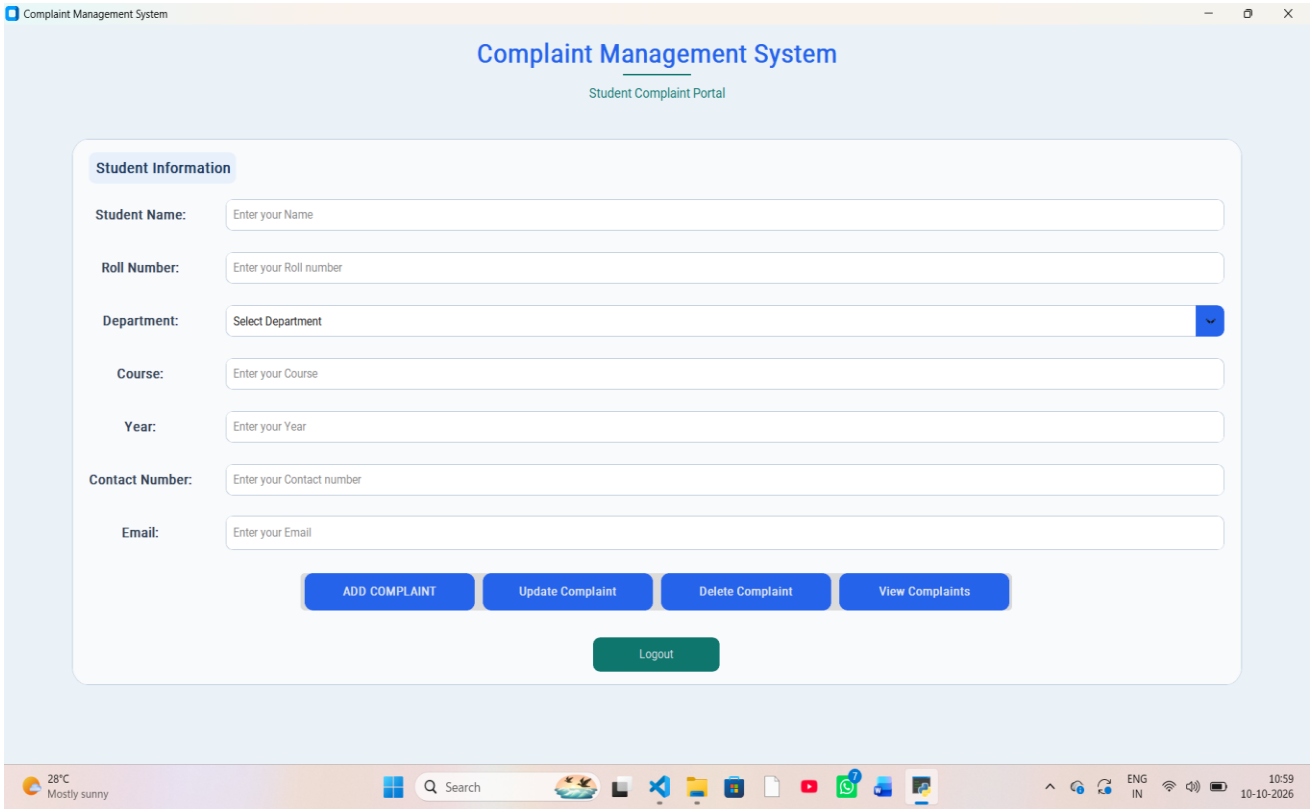
- CREATE TABLE: prepare the table structure when the database is first initialised.
- INSERT: add a new complaint when the user submits a valid form.
- SELECT: retrieve complaints for the list and fetch the details of a selected complaint.
- UPDATE: change supported fields such as status, assigned officer or remarks.
- DELETE: remove a record only if delete functionality is implemented and the user confirms the action.

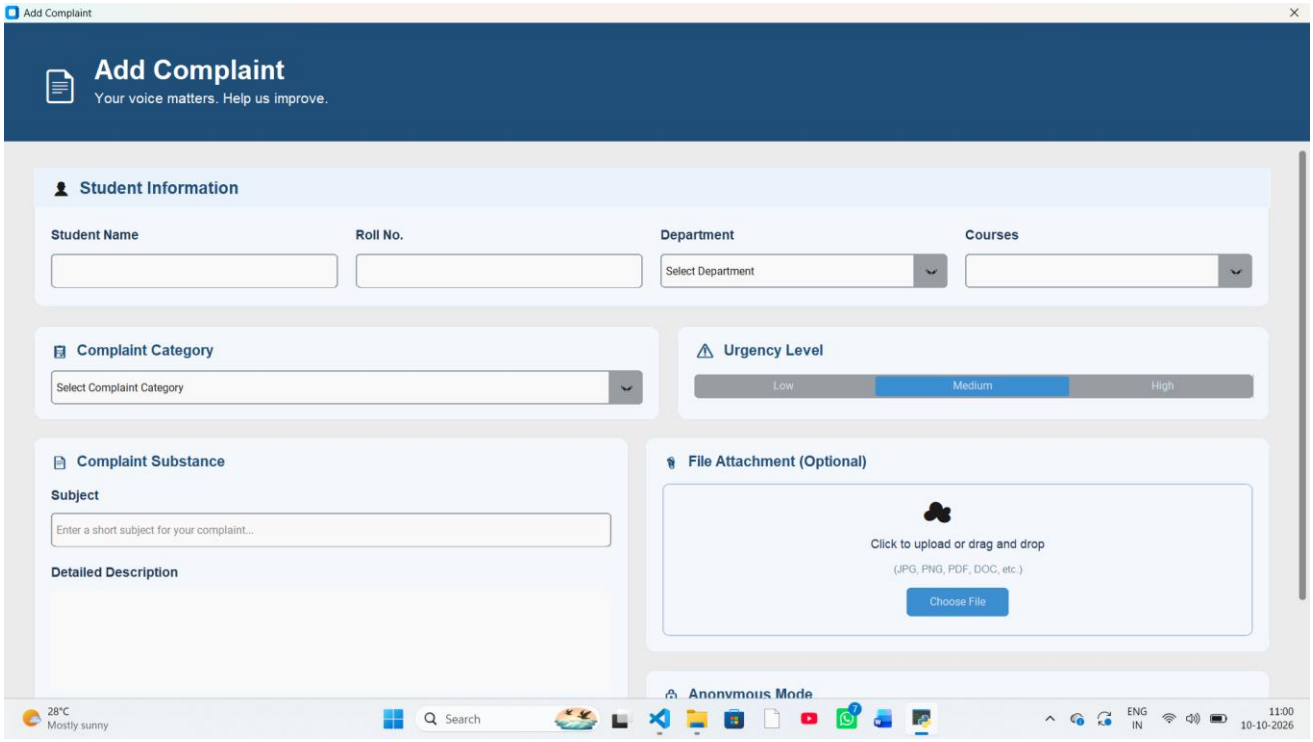
The existence of a button or dropdown does not, by itself, prove that the underlying database operation works. Each operation should be tested against the actual database.

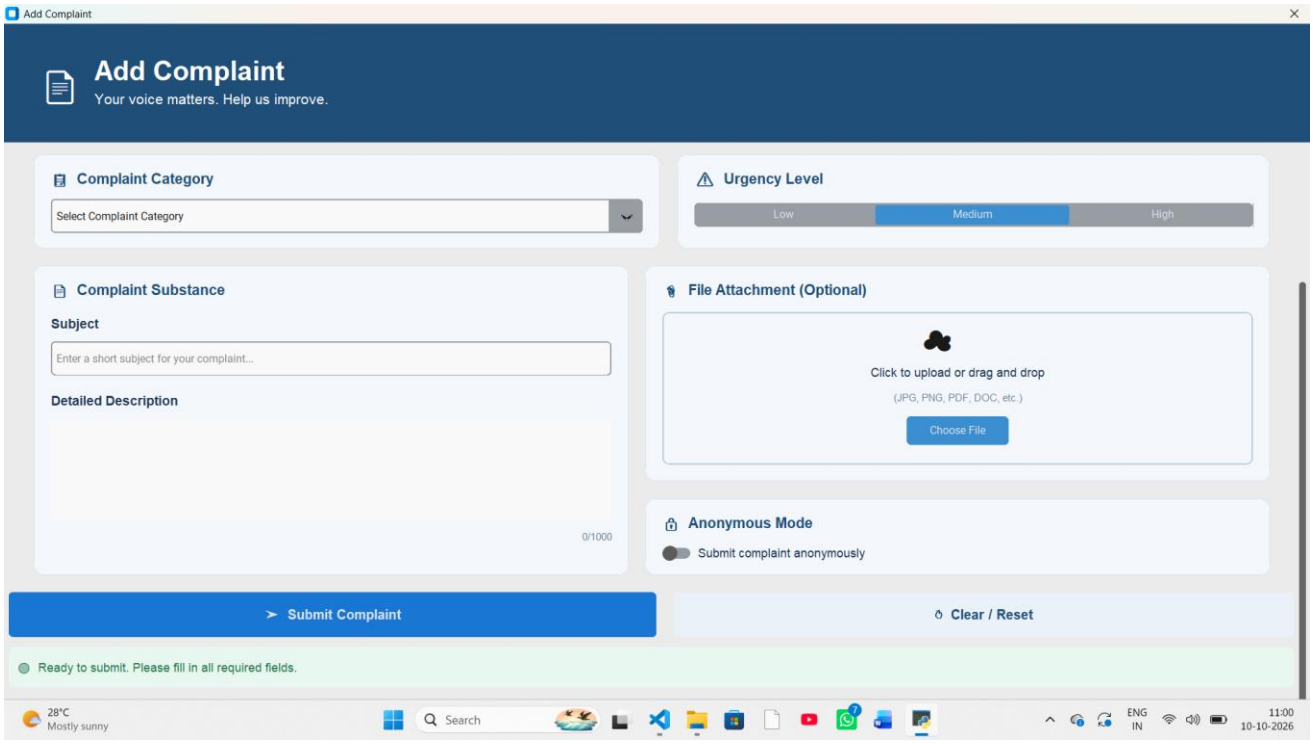
5.7 Screenshots

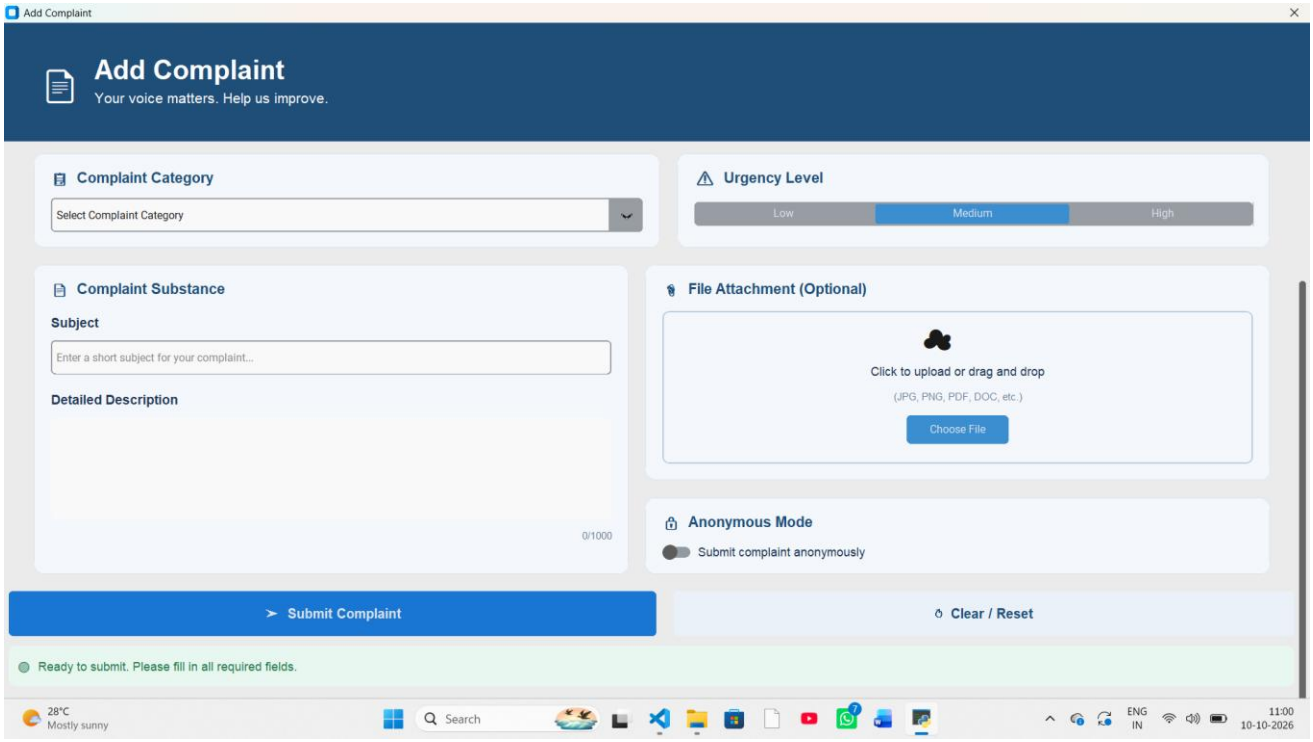


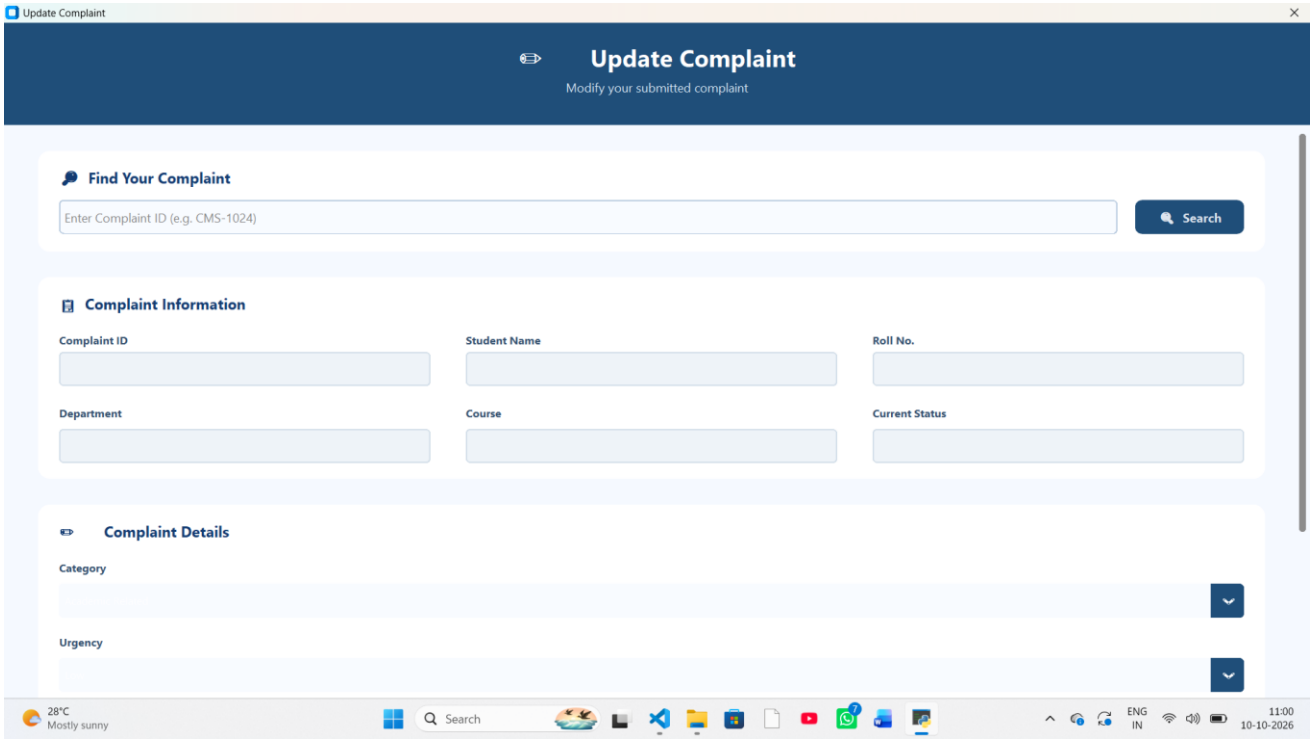
COMPLAINT MANAGEMENT SYSTEM

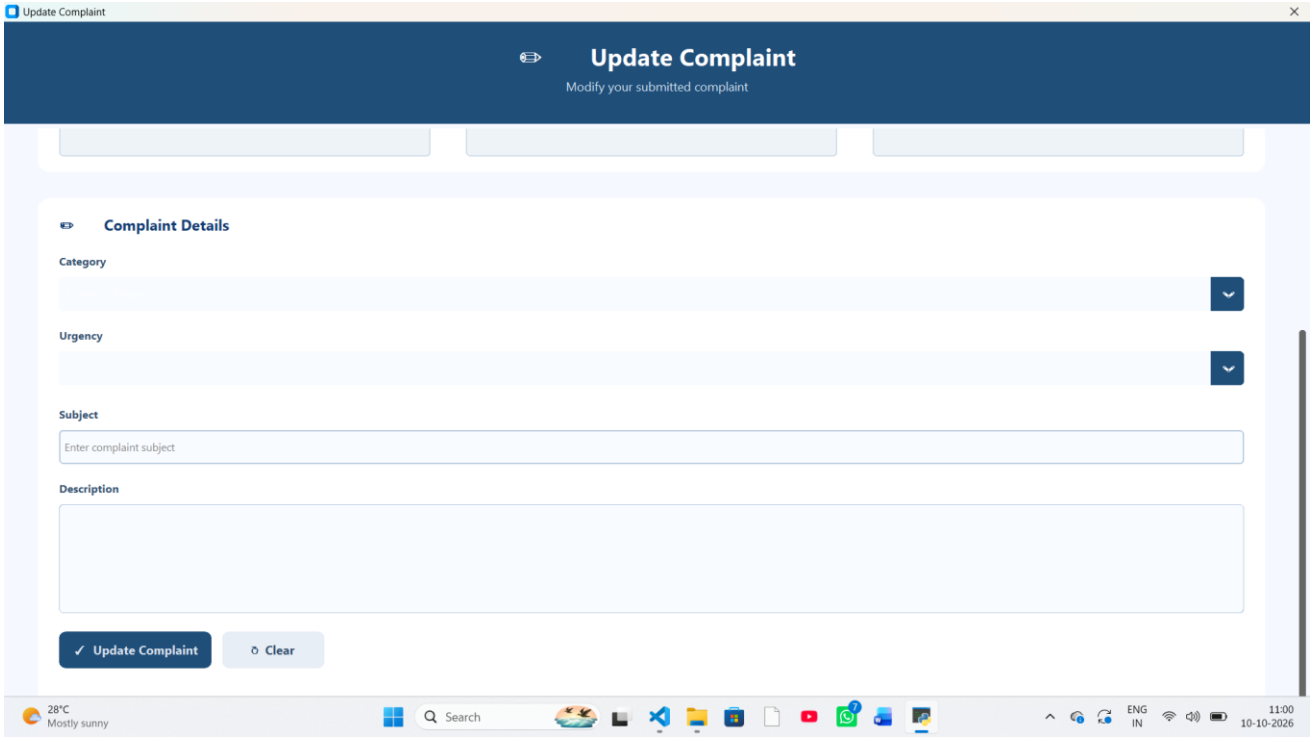


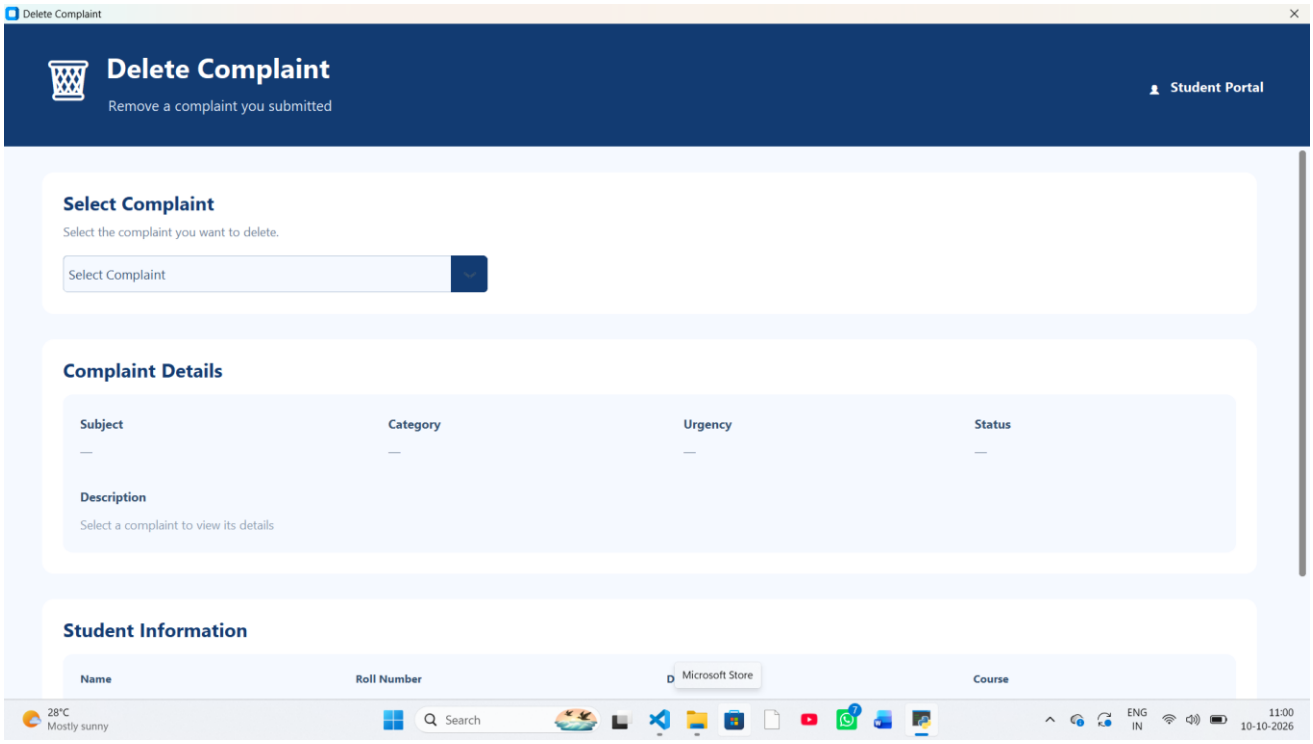


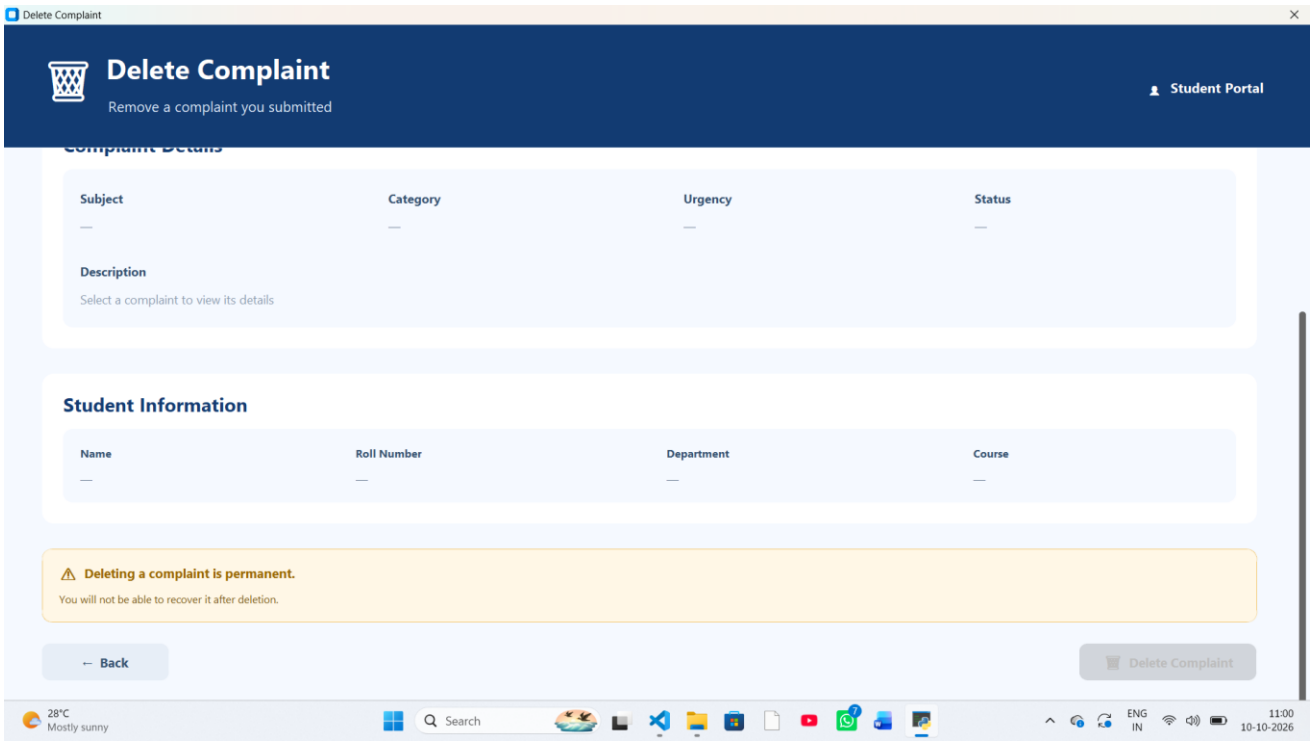

Add Complaint
Add Complaint
Your voice matters. Help us improve.
Complaint Category
Select Complaint Category
Urgency Level
Low
Medium
High
Complaint Substance
Subject
Enter a short subject for your complaint...
Detailed Description
0/1000
File Attachment (Optional)
Click to upload or drag and drop
(JPG, PNG, PDF, DOC, etc.)
Choose File
Anonymous Mode
Submit complaint anonymously
Submit Complaint
Clear / Reset
Ready to submit. Please fill in all required fields.
28°C
Mostly sunny
Search
ENG
IN
11:00
10-10-2026


Add Complaint
Add Complaint
Your voice matters. Help us improve.
Complaint Category
Select Complaint Category
Urgency Level
Low
Medium
High
Complaint Substance
Subject
Enter a short subject for your complaint...
Detailed Description
0/1000
File Attachment (Optional)
Click to upload or drag and drop
(JPG, PNG, PDF, DOC, etc.)
Choose File
Anonymous Mode
Submit complaint anonymously
Submit Complaint
Clear / Reset
Ready to submit. Please fill in all required fields.
28°C
Mostly sunny
Search
ENG
IN
11:00
10-10-2026

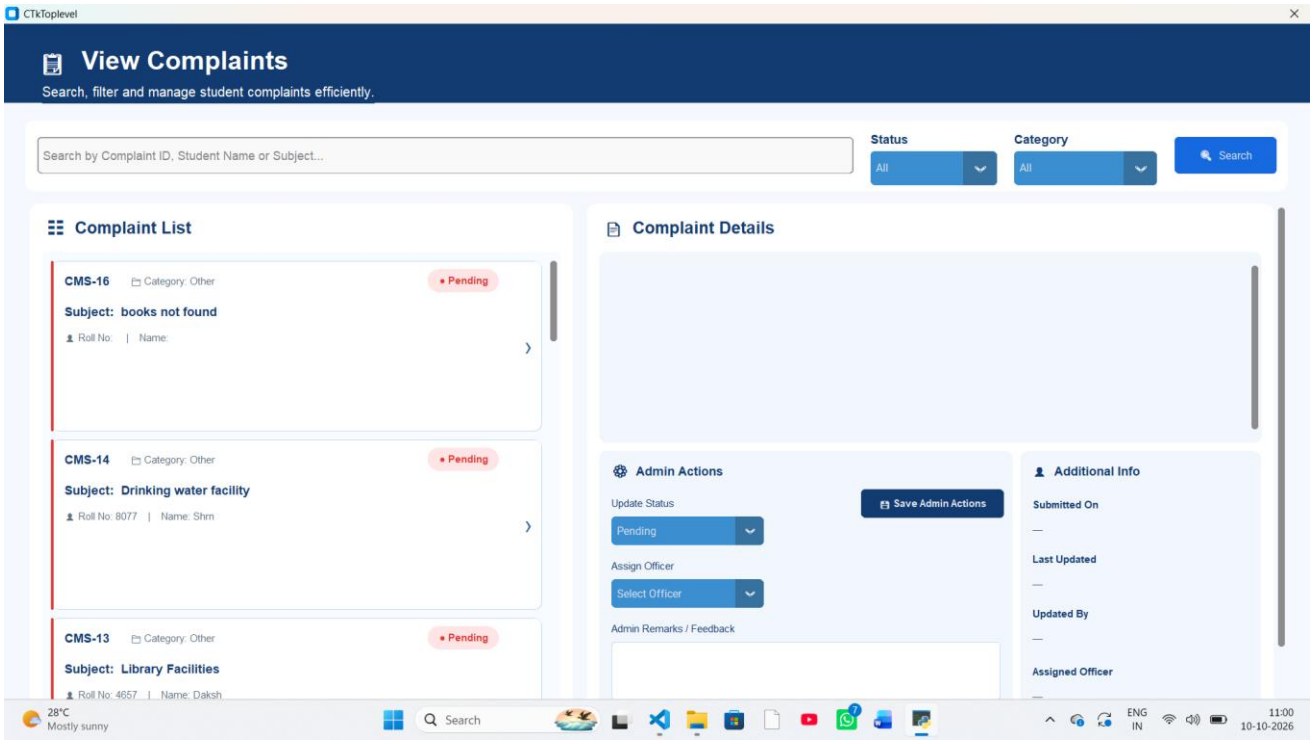

Update Complaint
Update Complaint
Modify your submitted complaint
Find Your Complaint
Enter Complaint ID (e.g. CMS-1024)
Search
Complaint Information
Complaint ID
Student Name
Roll No.
Department
Course
Current Status
Complaint Details
Category
Urgency
28°C
Mostly sunny
Search
ENG
IN
11:00
10-10-2026

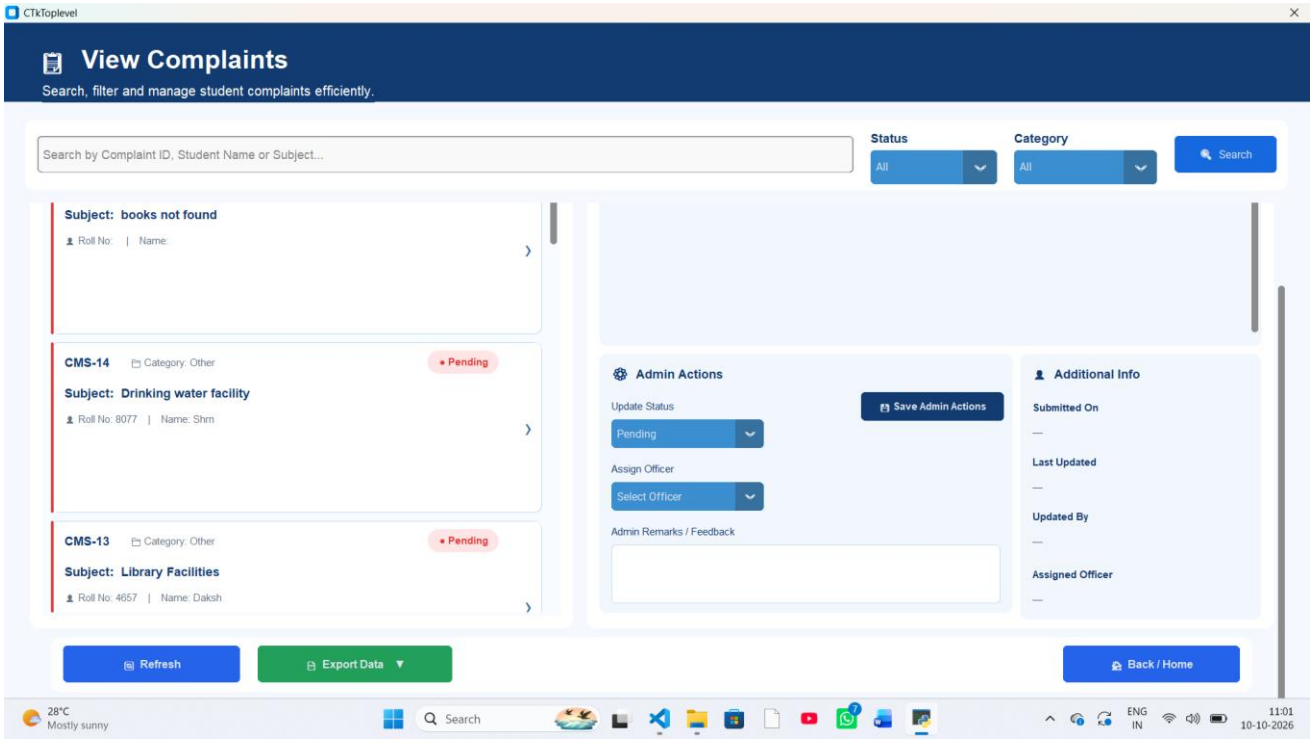

Update Complaint
Update Complaint
Modify your submitted complaint
Complaint Details
Category
Urgency
Subject
Enter complaint subject
Description
Update Complaint
Clear
28°C
Mostly sunny
Search
ENG
IN
11:00
10-10-2026


Delete Complaint
Delete Complaint
Remove a complaint you submitted
Student Portal
Select Complaint
Select the complaint you want to delete.
Select Complaint
Complaint Details
Subject
Category
Urgency
Status
Description
Select a complaint to view its details
Student Information
Name
Roll Number
Course
Microsoft Store
28°C
Mostly sunny
Search
ENG
IN
11:00
10-10-2026


Delete Complaint
Delete Complaint
Remove a complaint you submitted
Student Portal
Subject
Category
Urgency
Status
Description
Select a complaint to view its details
Student Information
Name
Roll Number
Department
Course
Deleting a complaint is permanent.
You will not be able to recover it after deletion.
← Back
Delete Complaint
28°C
Mostly sunny
Search
ENG
IN
11:00
10-10-2026

COMPLAINT MANAGEMENT SYSTEM





5.8 Running the Application

8. Install a compatible Python version and the packages imported by the project.
9. Keep the Python source files and the SQLite database path consistent.
10. Open the project folder in Visual Studio Code.
11. Run the application's main Python file.
12. Create a test complaint using clearly fictional example information.
13. Open the View Complaint screen and confirm that the record appears.
14. Close and reopen the application and verify that the record persists.

CHAPTER 6: PROJECT TESTING AND RESULTS

6.1 Introduction

Testing checks whether the application behaves as expected for normal use and for unusual or invalid input. Because this report is prepared from the project description rather than an execution log of the final source code, the table below is a test plan. Record the actual result after running each test; do not mark a test as passed without observing the behaviour.

6.2 Testing Approach

Testing Type	Purpose	Example
Unit testing	Check a small function or operation independently.	Test database connection or record retrieval.
Integration testing	Check that modules work together.	Submit a complaint and confirm it appears in View Complaints.
Interface testing	Check controls, layout and navigation.	Open windows, scroll, click cards and use dropdowns.
Validation testing	Check handling of incomplete or invalid values.	Submit a form with a required field blank.
Persistence testing	Check that stored data remains available.	Restart the app and reload records.
Regression testing	Check that a change has not broken an existing feature.	After changing the layout, repeat search and selection tests.

6.3 Test Cases

Test ID	Feature	Test Input / Action	Expected Result	Actual Status
TC-01	Launch the application	Run the main Python file.	The main window opens without an unhandled error.	To be tested
TC-02	Open Add Complaint	Click the Add Complaint button.	The complaint-entry window opens.	To be tested
TC-03	Submit valid complaint	Enter valid sample values and submit.	The record is saved and clear feedback is shown.	To be tested
TC-04	Required field missing	Leave a required field blank and submit.	The form prevents invalid submission or clearly reports the issue.	To be tested
TC-05	Database persistence	Submit a record, close and reopen the app.	The saved record remains available.	To be tested
TC-06	View complaint list	Open View Complaints.	Stored records appear in the list.	To be tested
TC-07	Select complaint card	Click a record in the list.	The matching details appear in the details panel.	To be tested
TC-08	Search by ID	Enter an existing complaint ID.	The matching record is shown.	To be tested
TC-09	Search by student or subject	Search using an existing name or subject.	Matching records are shown according to the search rules.	To be tested
TC-10	Status filter	Choose Pending, In Progress or Resolved.	Only records with the selected status are shown.	To be tested
TC-11	Category filter	Choose a category such	Only records in the	To be tested

		as Hostel or Academics.	selected category are shown.	
TC-12	No search results	Enter text that does not match a record.	The interface shows an empty state or no-results message.	To be tested
TC-13	Update status	Select a new status and save, if supported.	The changed status remains after reloading.	To be tested
TC-14	Assign officer / add remarks	Enter assignment and remarks, then save, if supported.	The values persist and are displayed when the record is reopened.	To be tested
TC-15	Delete complaint	Use delete flow only if implemented; confirm deletion.	The intended record is removed and unrelated records remain.	To be tested
TC-16	Window and scroll behaviour	Resize the window and scroll list/details.	Important controls and content remain accessible.	To be tested

6.4 Recording Results

After executing the tests, replace “To be tested” with the observed result, such as “Pass” or “Fail”, and add a short note for any failure. If a feature is not implemented, mark it “Not implemented” rather than “Pass”. A failed test should be corrected and rerun before the report is submitted.

6.5 Verification and Validation

Verification asks whether the software has been built according to its requirements and design. Validation asks whether it supports the intended user task. In this project, verification includes checking that the database fields, search controls and status values match the specification. Validation includes confirming that a user can enter a complaint, locate it later and understand its current recorded status.

6.6 Expected Result Summary

When correctly integrated, the application should provide a structured local workflow for complaint entry and review. The strongest evidence of successful implementation will be a genuine demonstration that a complaint submitted through the form is stored in SQLite, appears in the list, opens with the correct details, and remains available after restarting the application. Administrative updates should be reported as complete only after persistence has been tested.

CHAPTER 7: IMPLEMENTATION EVALUATION

7.1 Evaluation of the System

The project brings complaint information into a consistent form and provides a visual way to review stored records. A list-and-details layout allows a user to scan several complaints and inspect one selected record. Search and category/status filters are intended to reduce the time needed to locate a particular item.

The use of SQLite keeps the database lightweight and local, making it practical for a student project. CustomTkinter supports a desktop interface without requiring a browser or web server. These choices are suitable for demonstrating fundamental programming and database concepts.

7.2 Limitations

- The current application is a local desktop system; remote access is not part of the confirmed scope.
- A local database file does not automatically provide multi-user concurrency, central backups or institutional access controls.
- The report does not assume that status, assignment, remarks, update and delete controls are fully connected to database writes; these must be verified in the final source code.
- No automatic email, SMS, push notifications or escalation rules are confirmed.
- No formal security audit or user acceptance study has been documented here.
- Test outcomes are intentionally left for execution and recording against the final application.
- Screenshots, exact package versions and the final database schema must be added or confirmed before submission.

7.3 Future Scope

- Add role-based access for students, administrators and department staff.
- Implement robust input validation and clear success/error messages.
- Complete and verify database-backed update, assignment, remarks and delete workflows.
- Add timestamps and an audit trail showing when a complaint was updated and by whom.
- Add secure backup and restore options for the database.
- Introduce dashboards for complaint counts by status, category and date.
- Add notifications when a complaint is submitted or its status changes.
- Develop a web or mobile version for wider access, with authentication and server-side security.
- Add export options such as CSV or PDF reports, where appropriate.
- Conduct usability testing with representative users and document their feedback.

7.4 Maintenance

Maintenance includes correcting errors, adapting the application to updated Python or package versions, improving the interface, backing up the database and reviewing whether stored data remains accurate. Before any upgrade, create a copy of the database and test the change with sample records. Personal complaint data should not be used in screenshots or shared report files without permission.

z

APPENDIX A: USER GUIDE

A.1 Starting the Application

15. Open the project folder in Visual Studio Code or the configured Python environment.
16. Check that the required Python packages are installed.
17. Run the main application file used by the project.
18. If a login or sign-in screen is present, use only the credentials configured for your test environment.

A.2 Adding a Complaint

19. Open the Add Complaint window from the main window.
20. Enter the requested student and complaint information.
21. Choose the relevant category and enter a clear subject and description.
22. Review the entries before submitting.
23. Confirm that the application reports success and that the record is stored.

A.3 Viewing and Searching Complaints

24. Open View Complaints.
25. Use the search field to enter a supported value such as complaint ID, student name or subject.
26. Use the status and category dropdowns to narrow the list.
27. Select a complaint card to display its details.
28. If no records appear, clear the search field and set filters to All.

A.4 Administrative Fields

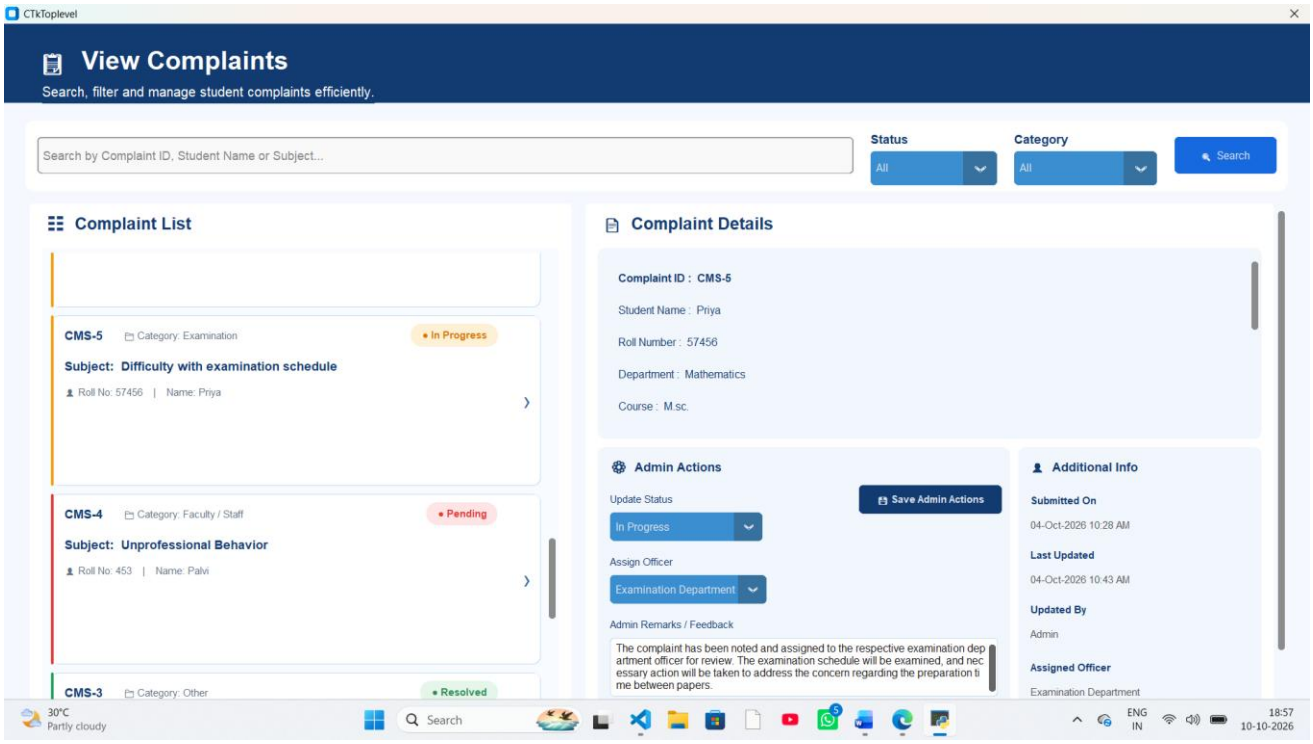
If the final version supports administrator actions, select the required status, choose an officer or department, enter remarks and use the appropriate save/update action. Reopen the record or restart the application to verify that the changes were saved. If no save action exists, the controls should be treated as interface-only until the functionality is implemented.

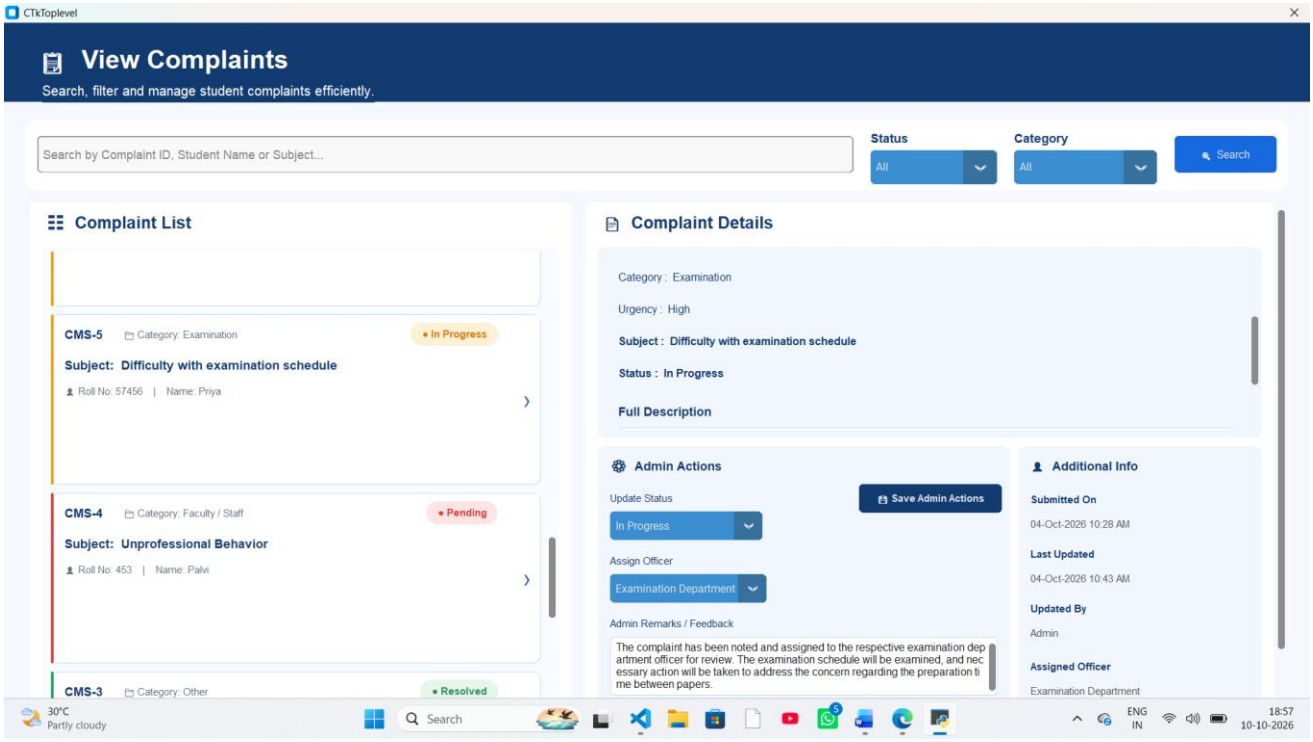
A.5 Safe Test Data

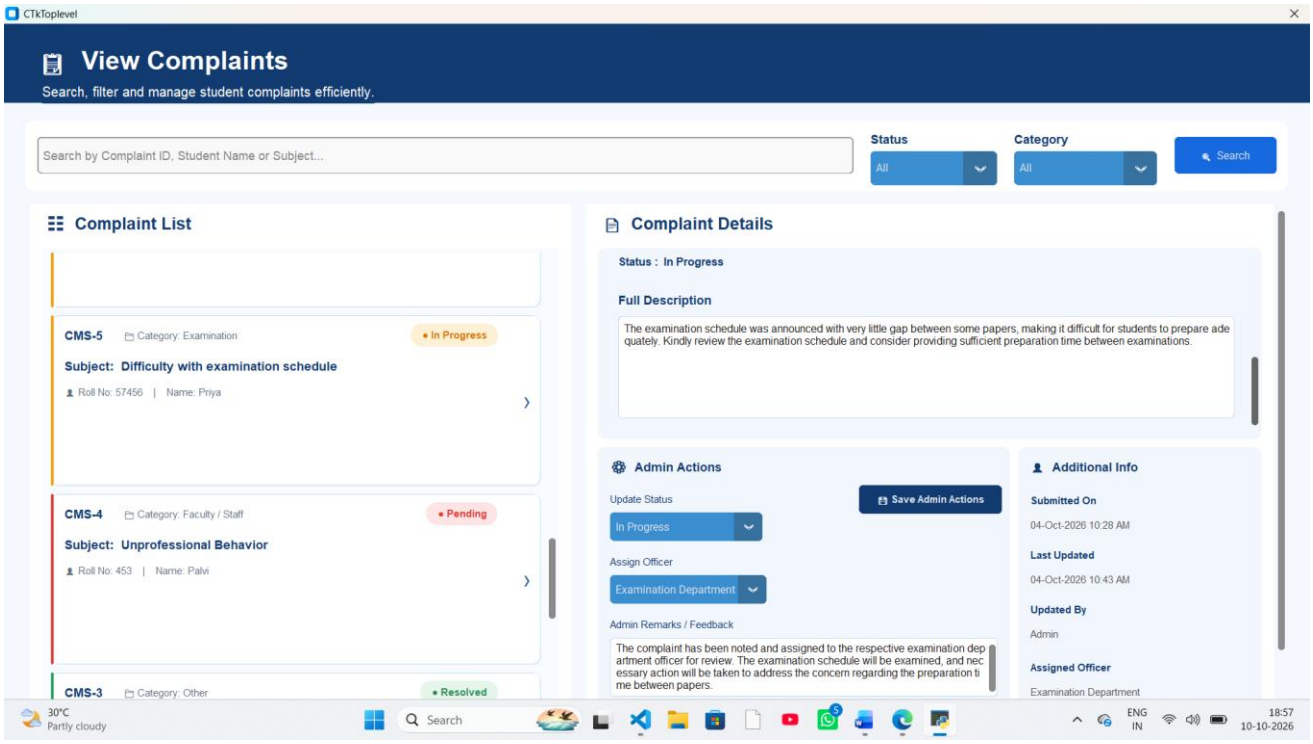
Use fictional sample records during demonstrations. Avoid including real personal complaint details, passwords or private information in screenshots submitted with the report.

APPENDIX B: SAMPLE COMPLAINT RECORD

The following example is illustrative and is included to demonstrate the type of information a complaint record may contain :






CTkToplevel
View Complaints
Search, filter and manage student complaints efficiently.
Search by Complaint ID, Student Name or Subject...
Status
All
Category
All
Search
Complaint List
CMS-5
Category: Examination
In Progress
Subject: Difficulty with examination schedule
Roll No: 57456 | Name: Priya
CMS-4
Category: Faculty / Staff
Pending
Subject: Unprofessional Behavior
Roll No: 453 | Name: Palvi
CMS-3
Category: Other
Resolved
Complaint Details
Status : In Progress
Full Description
The examination schedule was announced with very little gap between some papers, making it difficult for students to prepare adequately. Kindly review the examination schedule and consider providing sufficient preparation time between examinations.
Admin Actions
Update Status
In Progress
Save Admin Actions
Assign Officer
Examination Department
Admin Remarks / Feedback
The complaint has been noted and assigned to the respective examination department officer for review. The examination schedule will be examined, and necessary action will be taken to address the concern regarding the preparation time between papers.
Additional Info
Submitted On
04-Oct-2026 10:28 AM
Last Updated
04-Oct-2026 10:43 AM
Updated By
Admin
Assigned Officer
Examination Department
30°C
Partly cloudy
Search
ENG
IN
18:57
10-10-2026

CHAPTER 8: CONCLUSION

The Complaint Management System is an academic desktop application intended to make complaint recording and review more organised. Python provides the application logic, CustomTkinter provides the graphical interface, and SQLite provides local data storage. The project demonstrates how a real-world record-management problem can be represented through forms, a database and a set of user actions.

The principal value of the project is the integration of interface design, structured data storage and record retrieval in one application. A complaint-list and detail-view arrangement can make stored information easier to locate and understand. The search and filter controls are designed to support quicker navigation through records.

The final quality of the system depends on completing and verifying each database-backed action. In particular, complaint submission and retrieval should be tested end to end, while status changes, assignment, remarks, update and delete operations should be confirmed against the database. Genuine screenshots and actual test outcomes should be inserted before submission.

Overall, the project provides practical experience in Python GUI development, db Browser (SQLite) database operations, iterative development, software testing and technical documentation. It also offers a foundation for future enhancements such as stronger access control, audit history, notifications and multi-user access.

BIBLIOGRAPHY

- Python Software Foundation. Python Documentation. https://docs.python.org/3/
- Python Software Foundation. sqlite3 — DB-API 2.0 interface for SQLite databases. https://docs.python.org/3/library/sqlite3.html
- SQLite Consortium. SQLite Documentation. https://www.sqlite.org/docs.html
- CustomTkinter project documentation. https://customtkinter.tomschimansky.com/
- TkDocs. Tkinter documentation and tutorials. https://tkdocs.com/
- Visual Studio Code Documentation. https://code.visualstudio.com/docs